Learning System 2: Intro to Gradient Decent, Linear and Non-Linear Regression
DT8008 VT26, Halmstad University

Binay Kumar Sah, binsah25@student.hh.se, Master's in Embedded and Intelligence System
Anton Bauer, antbau25@student.hh.se, TAEIS 25

Feburary 2026

Introduction

This lab introduces the concept of regression and gradient descent which are also the fundamental techniques in machine learning. Through step-by-step implementations, we explore gradient descent for minimizing functions of one, two, and multiple parameters respectively. The lab progresses from linear regression with one feature to multiple features, and finally to nonlinear regression, using real-world datasets like housing prices. This hands-on approach helps in understanding optimization techniques and their applications in predictive modeling.

Optimization and Regression Technique

Part A - Gradient Descent

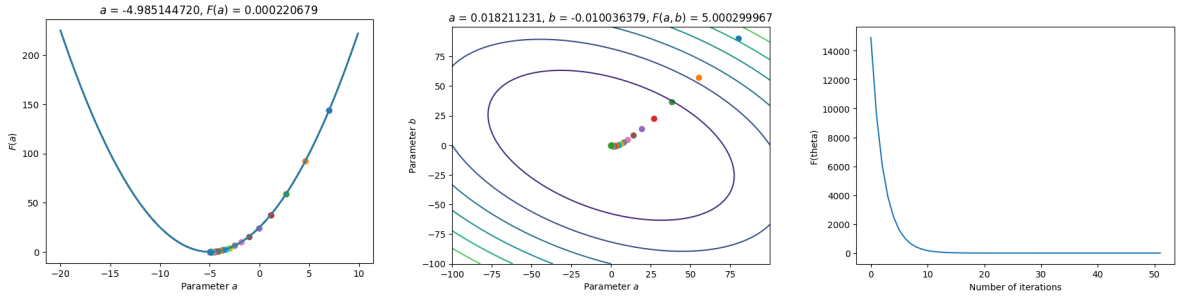
Objective

In this part, we were given to implement the gradient descent algorithm for one, two and multiple parameter in order to optimize the function. The goal was to iteratively update the parameters to find the minimum value of the function.

Key Concepts

- **Gradient Descent**: Gradient Descent is an optimization algorithm used to minimize a function by iteratively moving in the direction of the steepest descent (local minima).
- **Algorithm**:
  1. Initialize the parameters with some initial values.
  2. Set the learning rate α (e.g., $\alpha = 0.1$) and the convergence threshold ϵ (e.g., $\epsilon = 0.0001$).
  3. Repeat for a maximum number of iterations or until convergence:
     (a) Compute the gradient of the function $F(a)$ with respect to a i.e., $dF(a)$:
     (b) Update the parameter a using the gradient descent step:
     $$a = a - \alpha \cdot dF(a)$$
     (c) Check for convergence by evaluating the change in $F(a)$:
     $$\text{If } |F(a_{\text{prev}}) - F(a)| < \epsilon, \text{ stop.}$$
  4. Return the optimized parameter a.

- **Precautions**: If α is too small, it will take forever to get to the minimal point, and if it's too large, overstepping can occur
- **Observation and Results**: We implemented the gradient descent as per directed by the tasks and as we can see in the above figure 1a, 1b and 1c which are the output of GD implementation for the functions with one parameter $[F(a) = (a+5)^2]$, two parameters $[F(a,b) = 5+a^2+\frac{3}{2}b^2+ab]$ and multiple parameters $[F(\theta) = \sum_j \theta_j^2]$ respectively. From the results we observe that parameter a converges to -4.98 which is very close to expected -5. Likewise parameters a and b approaches to 0 for $F(a,b) = 5$ and for the multiple parameters $F(\theta)$ approaches to 0 which shows that our approximation was true as the the minimum of $F(\theta)$ is 0.



(a) Gradient descent for one parameter

(b) Gradient descent for two parameters

(c) Gradient descent for multiple parameters

Figure 1: Minimizing a function using Gradient descent

Part B: Linear regression with one feature

Objective

In part B, focus shifts from minimizing an already provided function, to applying linear regression with one feature. It is done in the context of gauging the profitability of opening a new restaurant in a city. The profitability is based on the reported earnings of food trucks and how populated the city is. Therefore, the one feature used is the population of a city, and the food trucks reported earnings can be used to find the linear relationship between city population and earnings.

New Key Concepts

- **Hypothesis**: A linear relationship is used as the hypothesis for the model: $h_\theta(x) = \theta^T x = \theta_0 + \theta_1 x_1$. Where θ_0 and θ_1 are the parameters and x_1 is the sole feature (population).
- **Cost Function**: In linear regression the aim is to find the parameters mentioned in the hypothesis by minimizing the error of the model (the fitted line). To measure the error a cost function is employed. The cost function used in our case is the mean squared error (MSE):

$$E(\theta) = \frac{1}{2n}\sum_{i=1}^{n}[h_\theta(x^{(i)}) - y^{(i)}]^2$$

- **Algorithm & Precautions**: Remain the same as described in part A.
- **Observation and Results**: We implemented the linear regression with one feature using gradient descent, as can be seen in Figure 2. Where the left plot shows the fitted line and corresponding parameters, showing how population affects predicted earnings. Right plot shows the number of iterations required before either max iterations, or convergence is reached. These plots were created with an $\alpha = 0.01$, $\epsilon = 0.00001$ and $max\_iterations = 5000$. The most notable observation

from the plots is that our model converged and only needed half the max iterations, and that there exists a gap in data for populations below 50000. The fitted line was then used to predict the earnings of a population of 35000 and 70000, yielding predicted earnings of 0.39284854340918196 and 4.507181347895143. Converting the earnings (10 000 dollars per unit), we estimated a dollar profit of $3 928 and $45 071. This is similar to what a well established framework like Scikit-learn predicted: 0.27983688 4.45545463, or $2 798 and $44 555. And as mentioned earlier, there exists a gap in the data for populations below 50 000; which explains the larger deviation on the 35 000 population data point as there simply are no close examples to refer to.

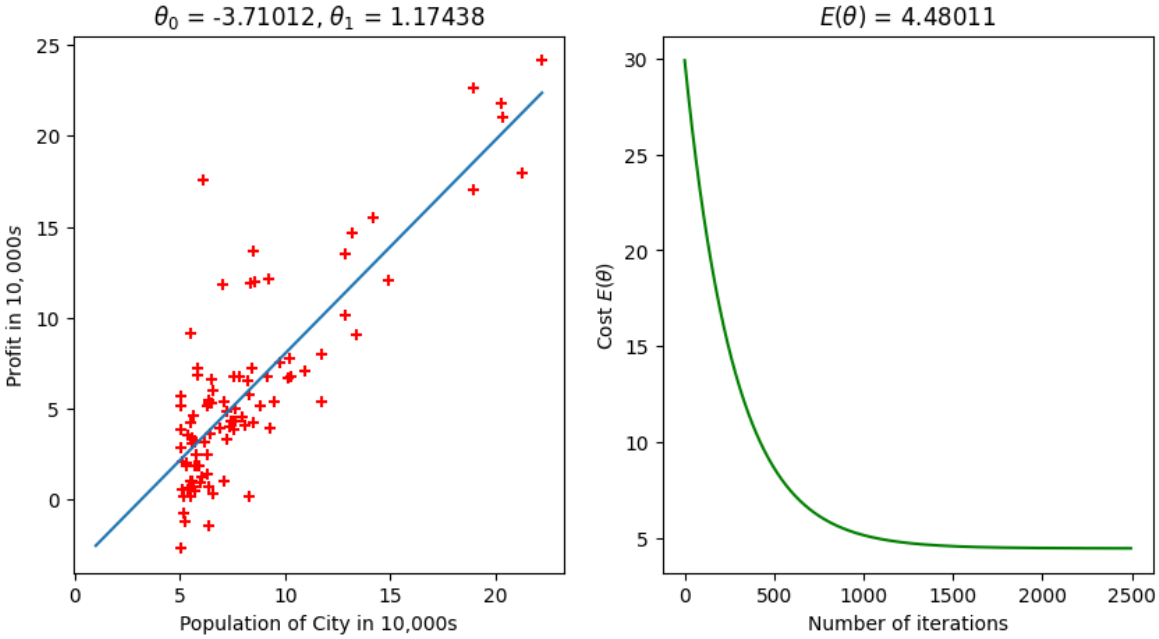


Figure 2: Linear regression with one feature using gradient descent. Left plot shows the fitted line and corresponding parameters, showing how population affects predicted earnings. Right plot shows the number of iterations required before either max iterations or convergence is reached. These plots were created with an $\alpha = 0.01$, $\epsilon = 0.00001$ and $max\_iterations = 5000$.

Part C: Linear regression with multiple features

Objective

In Part C, we continue working with the linear regression from Part B and extend it to work with multiple features. It is done in the context of housing prices, where the aim of the model is to try predict the price of a house given the 2 features: size of the house (in square feet), and the number of bedrooms.

New Key Concepts

- **Hypothesis**: The assumed linear relationship remains, but it now has to account for 2 features. As such, the hypothesis can be described as:

$$h_\theta(x) = \theta^T x = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

Where θ_0, θ_1 and θ_2 are the parameters, and the features x_1 for house size and x_2 for number of rooms.

- **Feature scaling**: Introducing more than one feature is problematic when the features vary greatly in range. In our case the number of rooms range from 2 - 5, whilst the house size can vary from around 200 000 - 700 000. This is undesirable as the path will become skewed towards the larger perceived gain (where error minimizes the most), which makes the path of the gradient descent become bouncy - as one of the directions will be stretched. The solution is to scale the features in accordance to their own range in such a way that no feature is unfairly favored. One way, and the way done in Part C, is to normalize the values using Z-score standardization. The idea is to look at each feature separately, and by looking at the standard deviation and mean, assign a Z-score that can only range from -4 to 4 (where each unit represent a standard deviation). This means that if a room is greatly larger than the rest of the rooms it will get a high Z-score (max 4), and correspondingly, if a room is of exceptionally large size it will also gain a larger Z-score (max 4). As such, there is no longer any attention given to the scale of the numbers themselves but rather how they vary relative to themselves, making the overall cost-function less stretched, and allowing for a more direct path when using gradient descent. The normalized value is calculated as following:

$$X\_normalized = \frac{X - \bar{X}}{\sigma}$$

  Where $X\_normalized$ is a matrix containing the normalized values, X is the feature matrix, $\bar{X}$ is the mean of the feature matrix calculated along the columns (i.e. mean of each feature), and σ is the standard deviation (SD) along the columns of the feature matrix (i.e. SD for each feature).

- **Algorithm**:

  1. Start by reading the data from the csv file, normalizing the features, and adding 1:s to account for the intercept.
  2. Set up hyperparameters (initial θ:s, max_iterations, ϵ, α/α:s)
  3. Perform gradient descent using MSE as cost function until either convergence, or max iterations, is reached.

- **Normal Equation**: Linear regression can be done without gradient descent with some prerequisites. In our case, as the cost function we use (MSE) is a convex function it follows that only one part point will contain the minimium error, and that point will be the only point to have a derivative of 0. As such, by taking the derivative of the cost function $E(\theta)$, and then setting it equal to 0, arriving at the following solution for θ:

$$\theta = (X^T X)^{-1} X^T y$$

  The benefit is that no feature scaling is required and the solution is given both directly and exactly; there is no need to loop until convergence or max iterations.

- **Precautions**: As stated in the section about feature scaling, it is important to normalize features that vastly differ. But, it is also crucial that a model trained on normalized values is only asked to predict on data points that have also undergone that same normalization, missing this will cause the model to predict very wrong.

- **Observation and Results**:

  1. **Data Normalization**: We performed the normalization on the dataset to ensure features (house size and number of bedrooms) were on a similar scale. Figure 3 is the scatter plots of the original and normalized datasets which showed that they are of same shape, but the normalized data was centered around the origin.

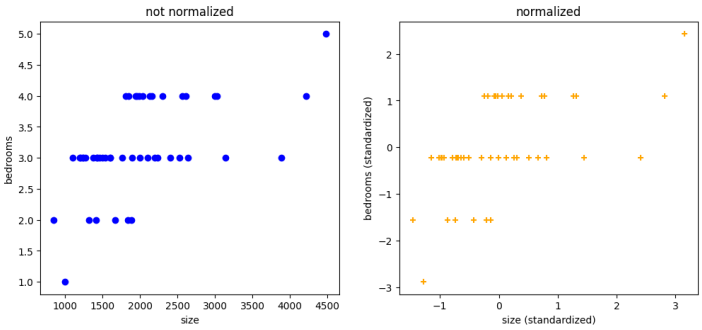


Figure 3: Normalized and not normalized data set

2. **Gradient Descent Implementation**: Here, linear regression was implemented using gradient descent with multiple features.We found that for ($\alpha = 0.01, 0.05, 0.1$), the cost converged quickly, with ($\alpha = 0.1$) being the fastest as shown in figure 4

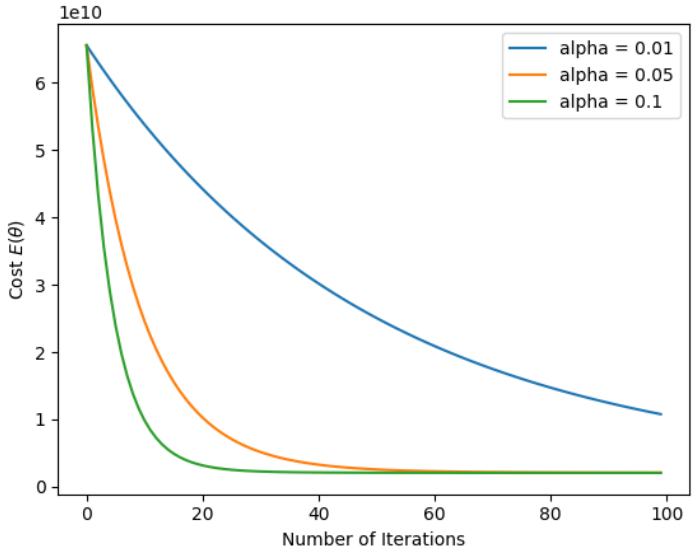


Figure 4: Normalized and not normalized data set

- **Prediction and Normal Equation**: Later we implimented the model successfully to predict the price of a 1650-square-foot house with 3 bedrooms, with results consistent across manual gradient descent and the normal equation. The normal equation provided a direct, non-iterative solution for (θ), yielding values similar to those obtained through gradient descent
- **Scikit-learn Implementation** : At end, we used the `LinearRegression` model from `scikit-learn` to perform the same task. The dataset was scaled using `StandardScaler`, and the model was trained on the normalized data. We found that predictions for test houses, such as a 1650-square-foot house with 3 bedrooms, were consistent with those from the manual implementation.

Part D: Nonlinear regression

Objective

In Part D, we learnt to implement the non linear regression using the Nadaraya-Watson estimator with a Gaussian kernel. The object was to predict the house price. We also explored the effect of the

hyperparameter γ on predictions and at last we compared the result with scikit-learn.

New Key Concepts

- **Nadaraya-Watson Estimator**:It is a method used in nonlinear regression to make predictions based on a weighted average of the outputs from training data. The weights are determined by a kernel function, such as the Gaussian kernel, which measures the similarity between the input data point and each training data point allowing the model to capture complex, nonlinear relationships in the data.
- **Gaussian Kernel**: It is a function used to measure the similarity between two data points in a nonlinear way. Mathematically,
$$k(u, v) = e^{-\gamma|u-v|^2}$$
where,
  - u and v are the two vector data points
  - γ is a hyperparameter that control the width of the kernel
- **Effect of Hyperparameter**:
  - **small** γ : Can lead to overfitting, resulting in too wiggly and noise
  - **large** γ: Can lead to underfitting, resulting model too flat and pattern ignorance
- **Algorithm**:
  1. **Input Definition:** Given training data $\{(x^{(i)}, y^{(i)})\}_{i=1}^n$, a query point x, and the bandwidth hyperparameter γ.
  2. **Kernel Computation:** Calculate the similarity between the test point and each training point using the Gaussian kernel:
$$k(x, x^{(i)}) = e^{-\gamma\|x-x^{(i)}\|^2} \tag{1}$$
  3. **Prediction (Hypothesis Function):** Compute the estimated output $h(x)$ via the weighted average:
$$h(x) = \frac{\sum_{i=1}^n k(x, x^{(i)}) \cdot y^{(i)}}{\sum_{i=1}^n k(x, x^{(i)})} \tag{2}$$
  4. **Handling Numerical Instability:** In the event the denominator $\sum k(x, x^{(i)}) = 0$:
     (a) Attenuate the bandwidth: $\gamma \leftarrow \gamma \times 0.1$.
     (b) Recompute $k(x, x^{(i)})$ and $h(x)$.
  5. **Model Evaluation:**
     (a) Apply the hypothesis function to the test dataset.
     (b) Compare predicted values against true outputs using error metrics.
     (c) Analyze the impact of γ by plotting predicted outputs versus varying bandwidth values.
- **Observation and Results**:
  1. **Implementation of the Gaussian Kernel**: At first, we implemented the Gaussian Kernel to find the similarity between the test data point x that is a house of 1650 square feet with 3 bedrooms with the given test data X. Then, using the hypothesis function, we have predicted the house price which turns out to be 303395.90574 as shown in figure 5

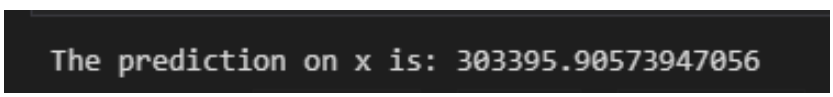


Figure 5: Price predition result for a new house of 1650-square-foor with 3 bedrooms

2. **Effects of Hyperparameter variation**:We also notice that the price prediction can vary according to the variation in γ. Figure 6 shows this variation, for small γ the predicted price didn't show much changes while the larger γleads to more variation.

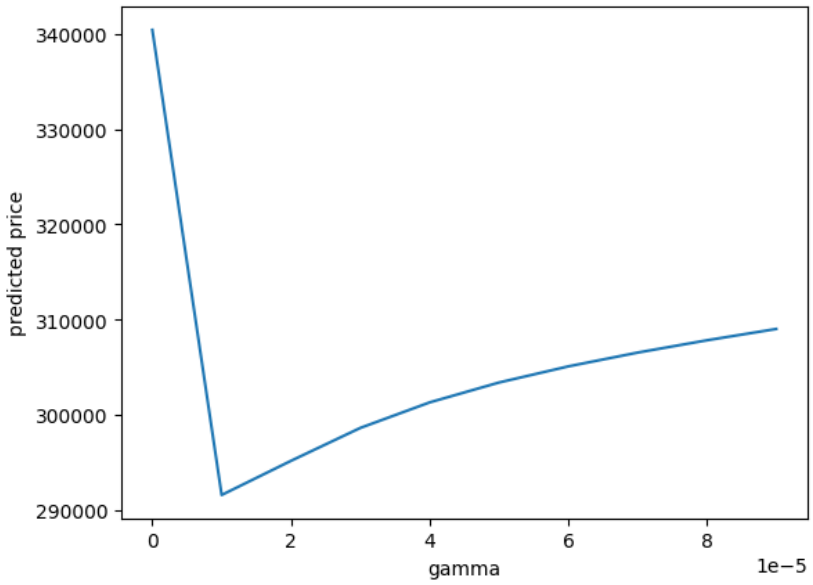
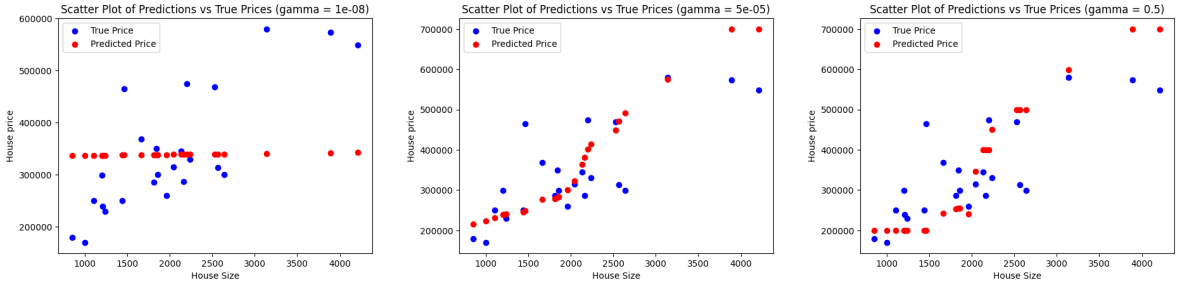


Figure 6: Variation in house price prediction with variation in γ

3. **Price Analysis**: After observing this variation we utilized the model to predict the price for the given data and then compare it with their actual price. For this we have split the data into two parts. First $(X\_train, 'y\_train')$ to train the model and other X_test as the test input to get y_pred which is then compared with the actual y_test. Figure 7 represent the scatter plotting of actual prices (blue) vs predicted price (red) for different value of γ. This shows smaller γ, the predicted price is closer to average price and shows less variation figure 7a, while for larger γ the predicted price align more closely with the actual price, indicating better prediction 7c



(a) At $\gamma = 0.00000001$ (b) At $\gamma = 0.00005$ (c) At $\gamma = 0.5$

Figure 7: Scatter Plot of Prediction vs True Prices at different γ

4. **Limitation of manual non linear regression model**: The above method was manual and was computationally expensive with harder to predict the accurate γ and analyze the result mathematically as we have to rely on the analytical observation.
5. **Nonlinear regression with scikit-learn (sklearn)**: This is where we implemented the same thing using `Scikit-learn` library. Here we used the `KernelRidge` model with an

RBF kernel to predict the house prices. The data splited and the featured were normalized using `StandardScaler`. After training the data the performance was evaluated using Mean Squared Error (MSE) (fig 8), which quantified the difference between predicted and true prices.This demonstrated the efficiency of scikit-learn in handling nonlinear regression tasks.

```
The MSE on the test data is: 2867062960.355849
```

Figure 8: MSE on a test data using Scikit-learn library

Conclusion

Thus lab 2 taught us a comprehensive understanding of regression techniques, from basic linear models to advanced nonlinear methods. It emphasized the importance of optimization, feature scaling, and hyperparameter tuning in building accurate predictive models. The manual implementations deepened our understanding of the underlying concepts, while scikit-learn demonstrated the efficiency and practicality of using machine learning libraries in real-world applications.